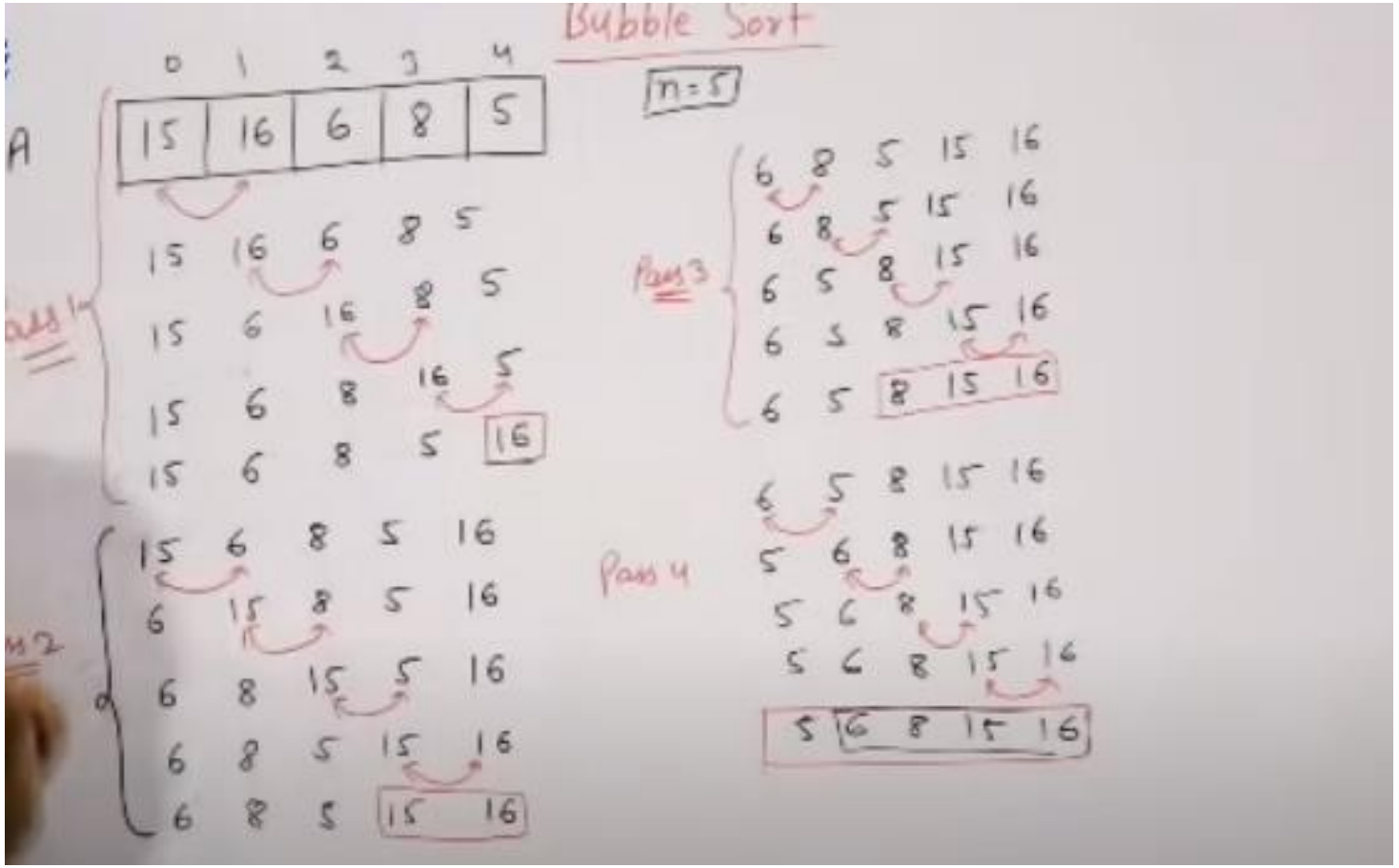
UNIT - 4

Sorting and Searching Algorithms and Hashing

Bubble sort

- Bubble sort is a simple sorting algorithm.
- This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.
- This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where **n** is the number of items.

Bubble sort



Bubble sort

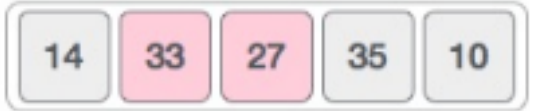
How Bubble Sort Works?

We take an unsorted array for our example. Bubble sort takes $O(n^2)$ time so we're keeping it short and precise.

Bubble sort starts with very first two elements, comparing them to check which one is greater.

In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.

We find that 27 is smaller than 33 and these two values must be swapped.



Bubble sort

The new array should look like this –

14	27	33	35	10

Next we compare 33 and 35. We find that both are in already sorted positions.

14	27	33	35	10

Then we move to the next two values, 35 and 10.

14	27	33	35	10

We know then that 10 is smaller 35. Hence they are not sorted.

14	27	33	35	10

We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –

14	27	33	10	35

Bubble sort

To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –

14	27	10	33	35

Notice that after each iteration, at least one value moves at the end.

14	10	27	33	35

And when there's no swap required, bubble sorts learns that an array is completely sorted.

10	14	27	33	35

Now we should look into some practical aspects of bubble sort.

Bubble sort

Algorithm

We assume **list** is an array of **n** elements. We further assume that **swap** function swaps the values of the given array elements.

```
begin BubbleSort(list)

   for all elements of list
      if list[i] > list[i+1]
         swap(list[i], list[i+1])
      end if
   end for

   return list

end BubbleSort
```

Worst case **time complexity** of Bubble Sort is **O(n^2)**.

Best case **time complexity** of Bubble Sort is **O(n)**.

Selection sort

- Selection sort is conceptually the most simplest sorting algorithm. This algorithm will first find the **smallest** element in the array and swap it with the element in the **first** position, then it will find the **second smallest** element and swap it with the element in the **second** position, and it will keep on doing this until the entire array is sorted.
- It is called selection sort because it repeatedly **selects** the next-smallest element and swaps it into the right place.

Selection sort

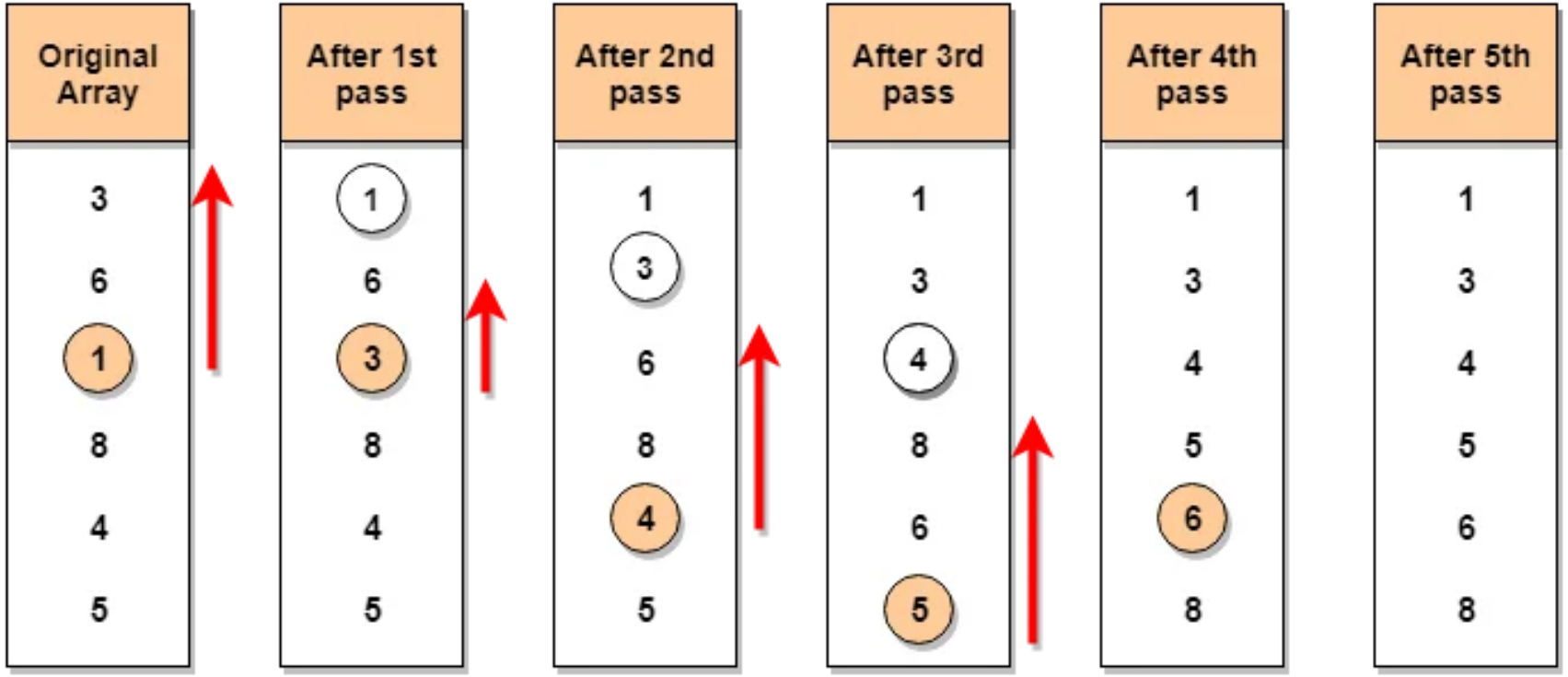
How Selection Sort Works?

Following are the steps involved in selection sort(for sorting a given array in ascending order):

1. Starting from the first element, we search the smallest element in the array, and replace it with the element in the first position.
2. We then move on to the second position, and look for smallest element present in the subarray, starting from index `1`, till the last index.
3. We replace the element at the **second** position in the original array, or we can say at the first position in the subarray, with the second smallest element.
4. This is repeated, until the array is completely sorted.

Selection sort

Let's consider an array with values {3, 6, 1, 8, 4, 5}



In the **first** pass, the smallest element will be 1, so it will be placed at the first position.

Then leaving the first element, **next smallest** element will be searched, from the remaining elements. We will get 3 as the smallest, so it will be then placed at the second position.

Then leaving 1 and 3(because they are at the correct position), we will search for the next smallest element from the rest of the elements and put it at third position and keep doing this until array is sorted.

Selection sort

Hence for a given input size of n,

Worst Case Time Complexity : $\mathbf{O(n^2)}$

Best Case Time Complexity : $\mathbf{O(n^2)}$

Selection sort

Step 1 – Set MIN to location 0

Step 2 – Search the minimum element in the list

Step 3 – Swap with value at location MIN

Step 4 – Increment MIN to point to next element

Step 5 – Repeat until list is sorted

Insertion sort

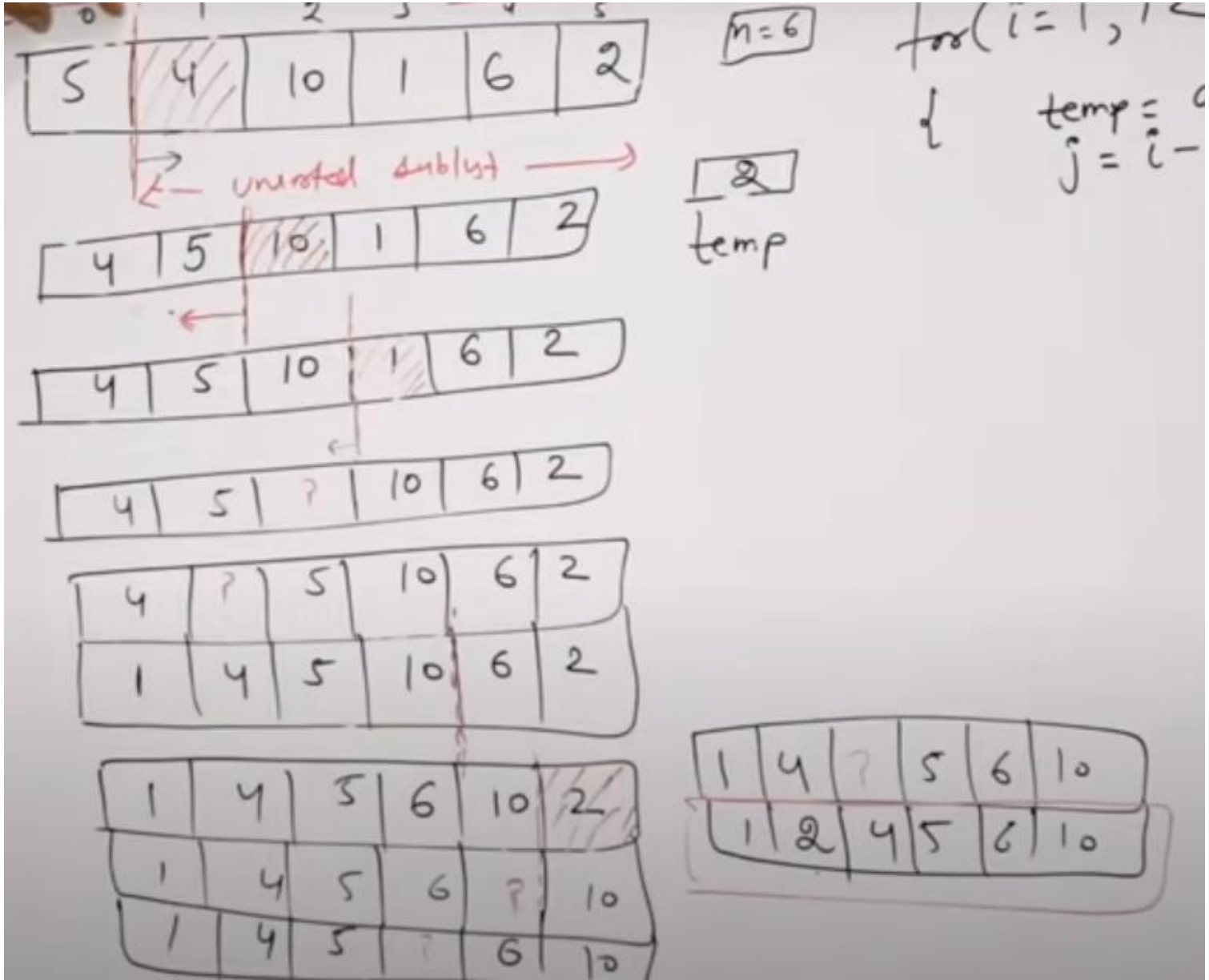
- Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed at the right side; otherwise, it will be placed at the left side. Similarly, all unsorted cards are taken and put in their exact place.

Insertion sort

Algorithm

- The simple steps of achieving the insertion sort are listed as follows -
- **Step 1 -** If the element is the first element, assume that it is already sorted. Return 1.
- **Step2 -** Pick the next element, and store it separately in a **key.**
- **Step3 -** Now, compare the **key** with all elements in the sorted array.
- **Step 4 -** If the element in the sorted array is smaller than the current element, then move to the next element. Else, shift greater elements in the array towards the right.
- **Step 5 -** Insert the value.
- **Step 6 -** Repeat until the array is sorted.

Insertion sort



Insertion sort

•**Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of insertion sort is **O(n)**.
•**Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of insertion sort is **$O(n^2)$**.
•**Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of insertion sort is **$O(n^2)$**.

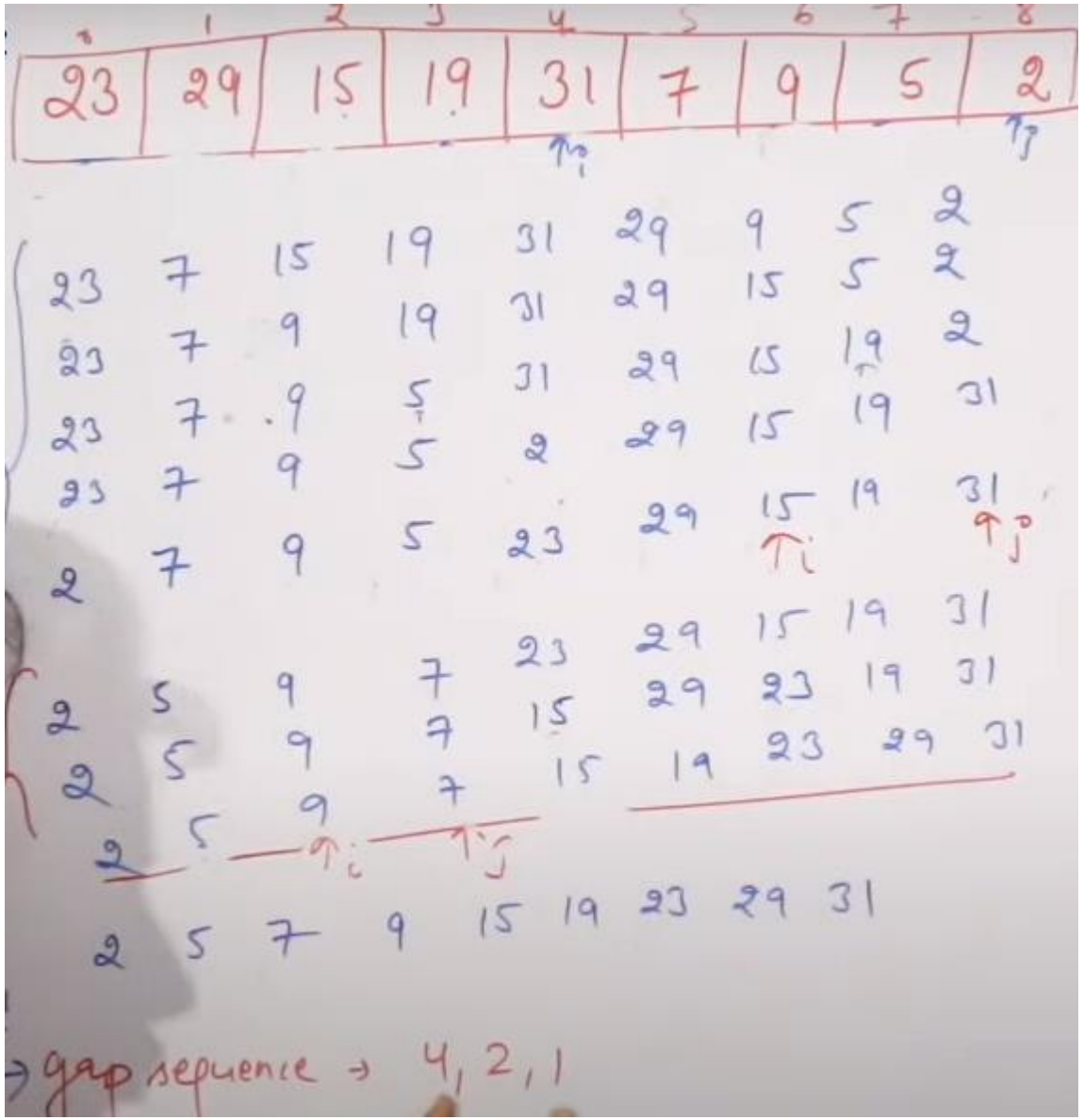
Shell sort

- It is a sorting algorithm that is an extended version of insertion sort. Shell sort has improved the average time complexity of insertion sort. As similar to insertion sort, it is a comparison-based and in-place sorting algorithm. Shell sort is efficient for medium-sized data sets.
- In insertion sort, at a time, elements can be moved ahead by one position only. To move an element to a far-away position, many movements are required that increase the algorithm's execution time. But shell sort overcomes this drawback of insertion sort. It allows the movement and swapping of far-away elements as well.
- We use shell sort when smaller elements are at last position of the array

Shell sort

- Step 1 − Start
  Step 2 − Initialize the value of gap size. Example: h
  Step 3 − Divide the list into smaller sub-part. Each must have equal intervals to h
  Step 4 − Sort these sub-lists using insertion sort
  Step 5 – Repeat this step 2 until the list is sorted.
  Step 6 – Print a sorted list.
  Step 7 – Stop.

Shell sort


0 1 2 3 4 5 6 7 8
23 29 15 19 31 7 9 5 2
i j
23 7 15 19 31 29 9 5 2
23 7 9 19 31 29 15 5 2
23 7 9 5 31 29 15 19 2
23 7 9 5 2 29 15 19 31
23 7 9 5 23 29 15 19 31
i j
2 7 9 7 23 29 15 19 31
2 5 9 7 15 29 23 19 31
2 5 9 7 15 19 23 29 31
2 5 9 7
i j
2 5 7 9 15 19 23 29 31
gap sequence → 4, 2, 1

Radix Sort Algorithm

<u>The steps used in the sorting of radix sort are listed as follows -</u>

- First, we have to find the largest element (suppose **max**) from the given array. Suppose **'x'** be the number of digits in **max**. The **'x'** is calculated because we need to go through the significant places of all elements.
- After that, go through one by one each significant place. Here, we have to use any stable sorting algorithm to sort the digits of each significant place.

Radix Sort Algorithm

181	289	390	121	145	736	514	212

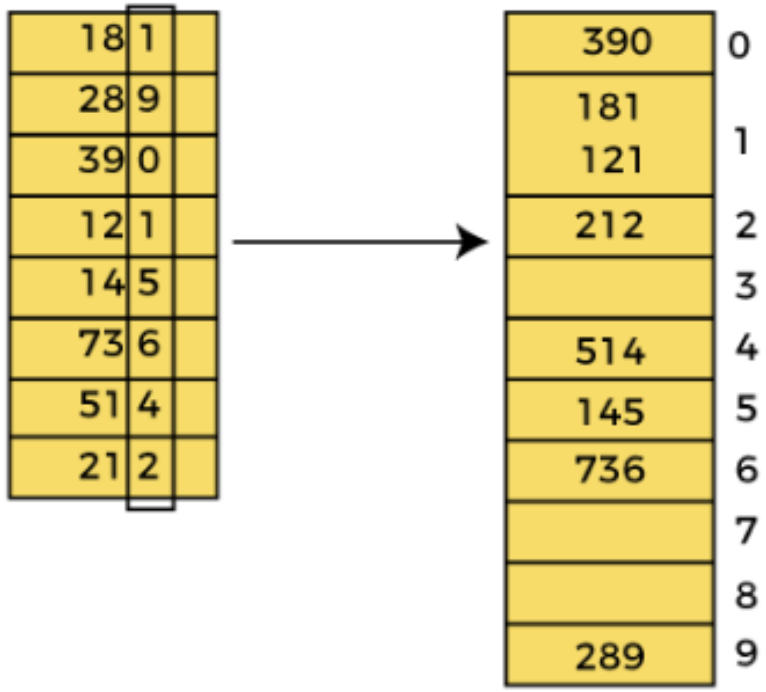
In the given array, the largest element is **736** that have **3** digits in it. So, the loop will run up to three times (i.e., to the **hundreds place**). That means three passes are required to sort the array.

Now, first sort the elements on the basis of unit place digits (i.e., **x** = **0**). Here, we are using the counting sort algorithm to sort the elements.

Radix Sort Algorithm

Pass 1:

In the first pass, the list is sorted on the basis of the digits at 0's place.



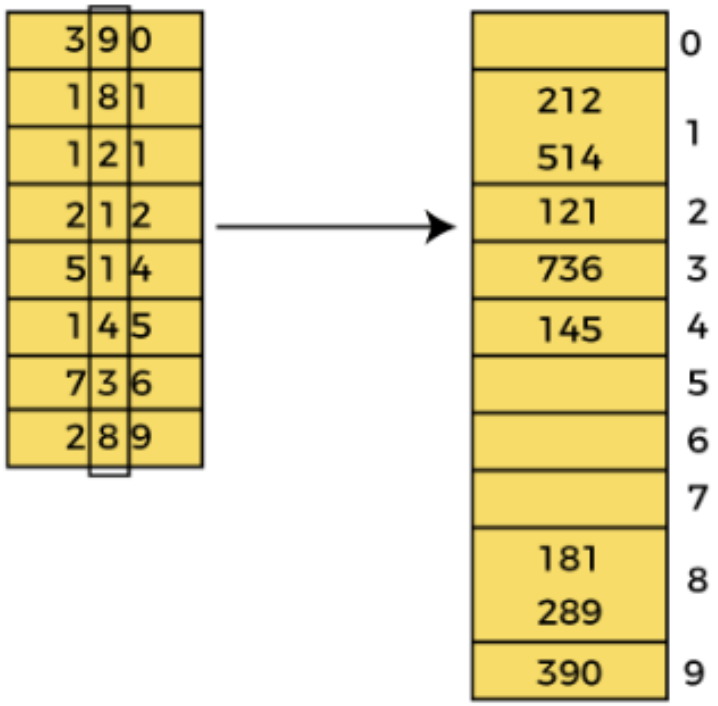
After the first pass, the array elements are -

390	181	121	212	514	145	736	289

Radix Sort Algorithm

Pass 2:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 10th place).



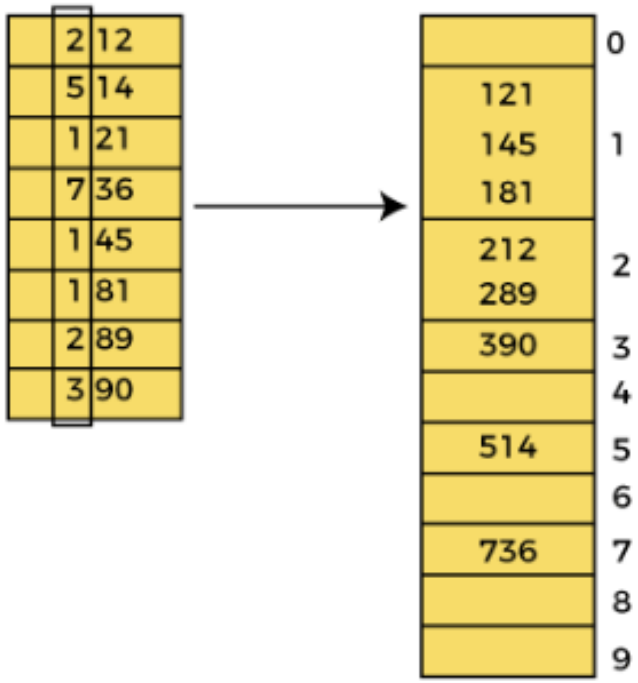
After the second pass, the array elements are -

212	514	121	736	145	181	289	390

Radix Sort Algorithm

Pass 3:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 100th place).



After the third pass, the array elements are -

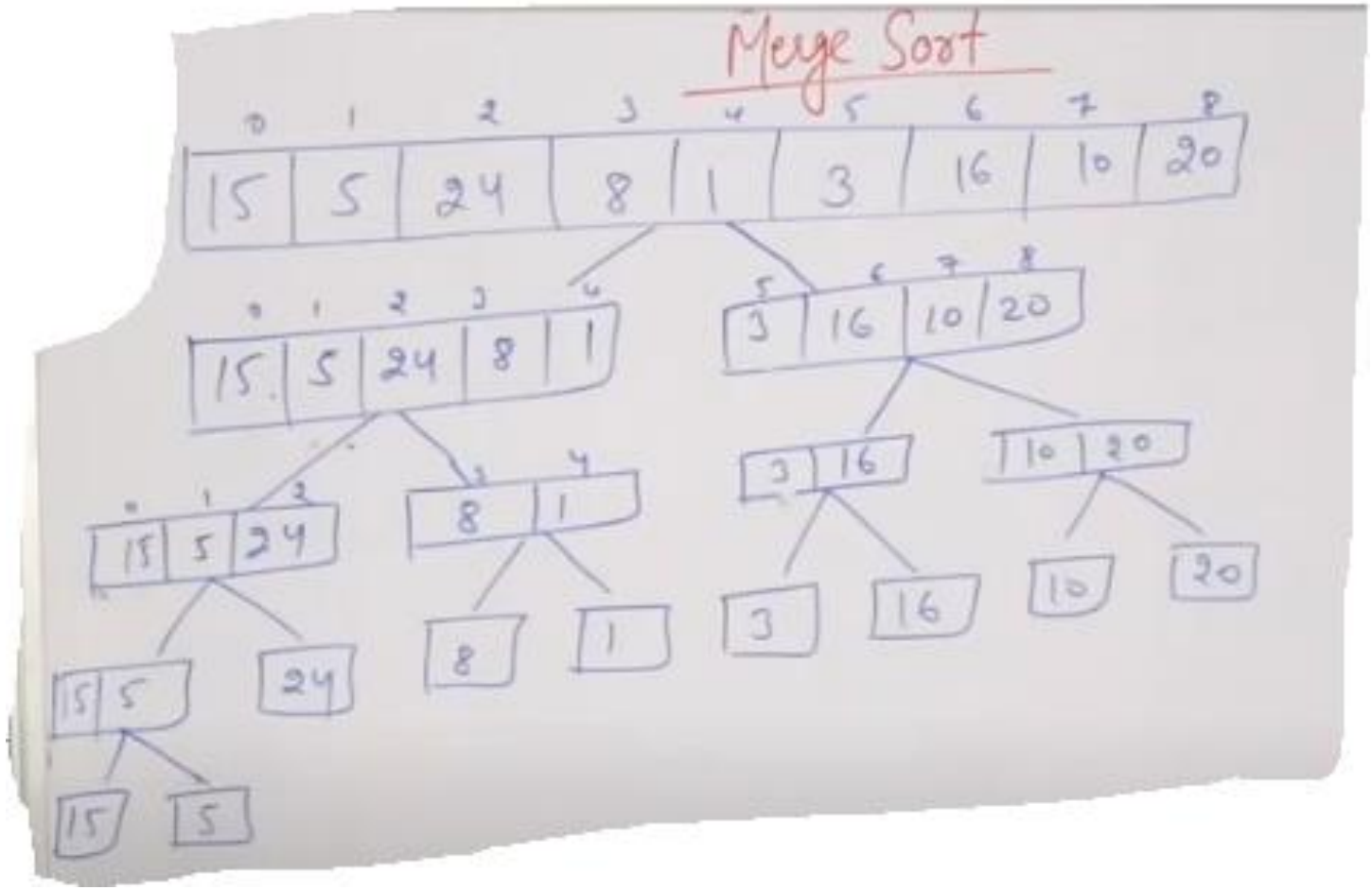
121	145	181	212	289	390	514	736

Merge sort

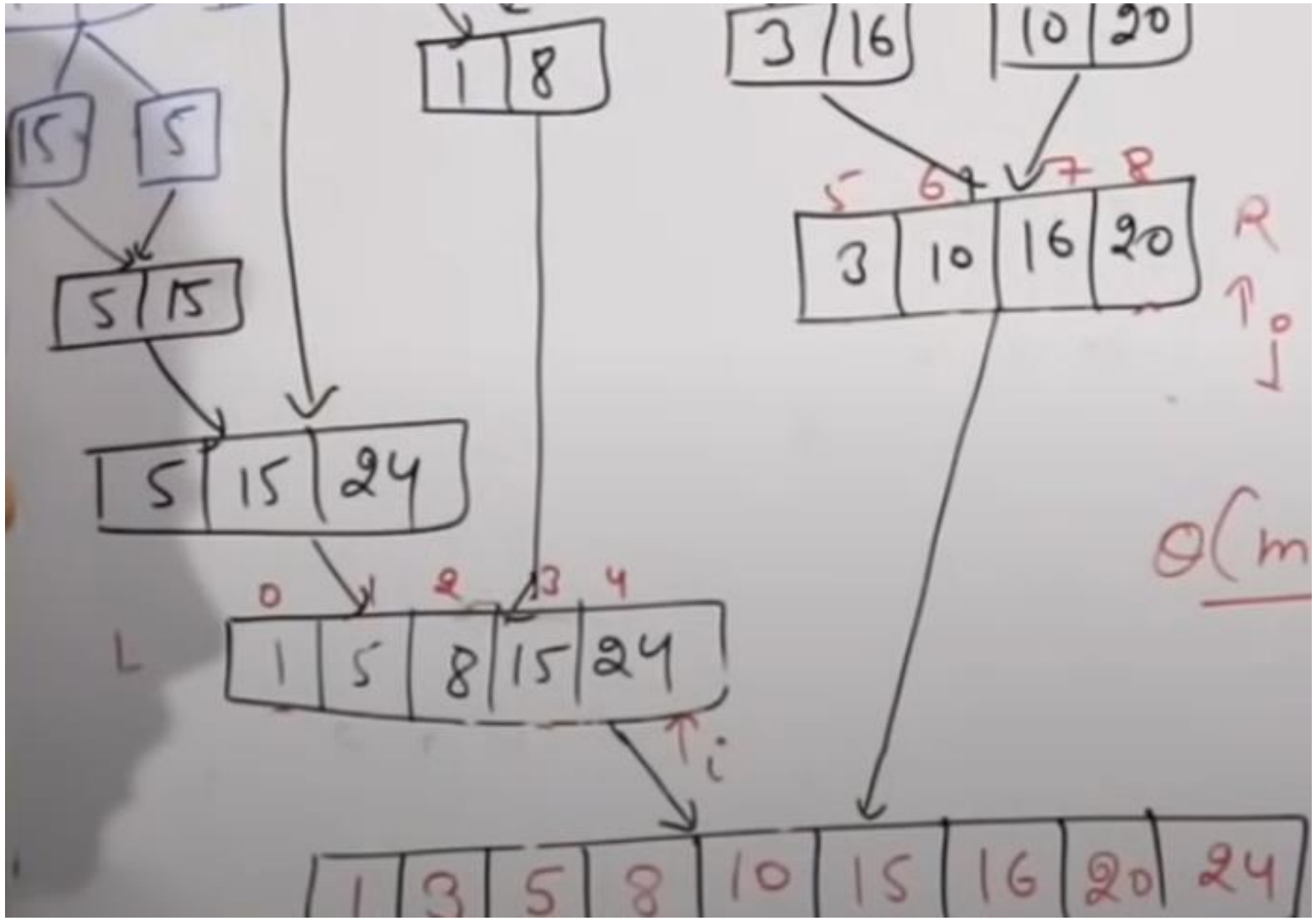
- Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being O(n log n), it is one of the most respected algorithms.
- Merge sort first divides the array into equal halves and then combines them in a sorted manner.

Worst Case Time Complexity : **O(n*log n)**
Best Case Time Complexity: **O(n*log n)**
Average Time Complexity : **O(n*log n)**

Merge sort



Merge sort

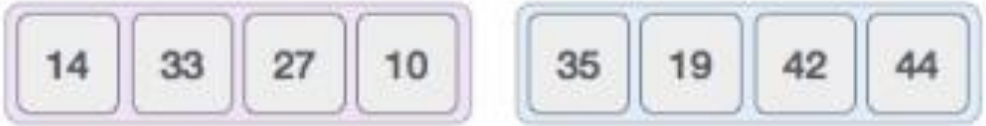

15
5
5 15
5 15 24
1 8
3 16
10 20
5 6 7 8
3 10 16 20
R
j
0 1 2 3 4
L
1 5 8 15 24
i
1 3 5 8 10 15 16 20 24

Merge sort

How Merge Sort Works?

To understand merge sort, we take an unsorted array as the following –

We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.

Merge sort

We further divide these arrays and we achieve atomic value which can no more be divided.

Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.

In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.

After the final merging, the list should look like this –

Quick Sort Algorithm

- This algorithm follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.
- **Divide:** In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.
- **Conquer:** Recursively, sort two subarrays with Quicksort.
- **Combine:** Combine the already sorted array.

Quick Sort Algorithm

- Divide, conquer and combine
- Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.
- After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.

Quick Sort Algorithm

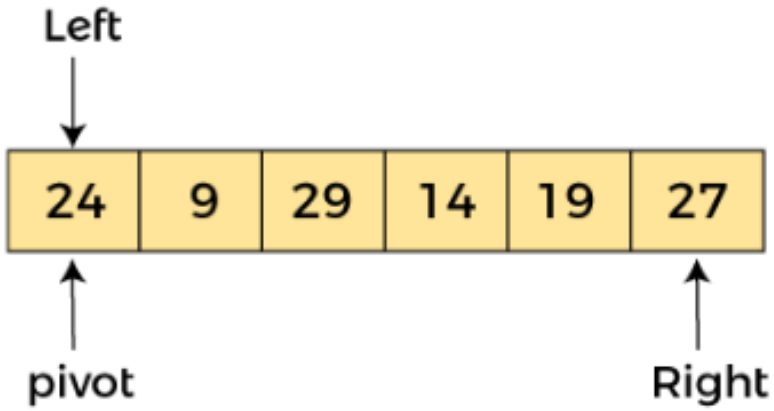
To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

Let the elements of array are -

24	9	29	14	19	27

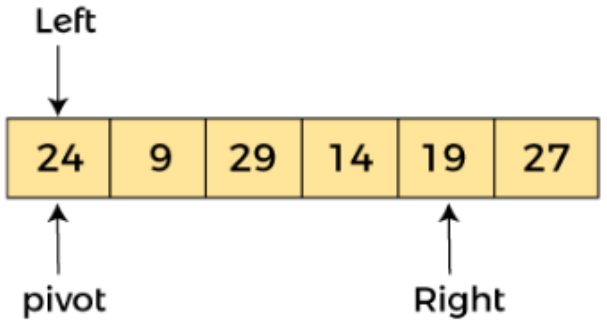
In the given array, we consider the leftmost element as pivot. So, in this case, a[left] = 24, a[right] = 27 and a[pivot] = 24.

Since, pivot is at left, so algorithm starts from right and move towards left.



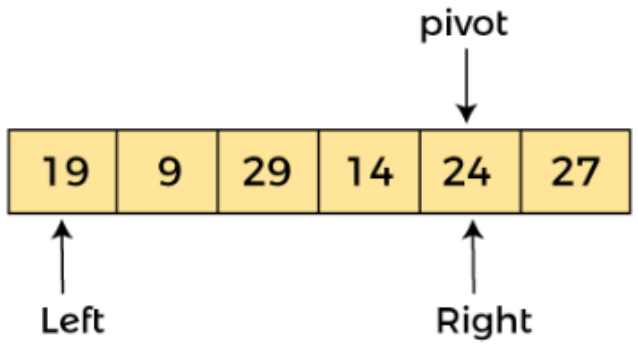
Quick Sort Algorithm

Now, a[pivot] < a[right], so algorithm moves forward one position towards left, i.e. -



Now, a[left] = 24, a[right] = 19, and a[pivot] = 24.

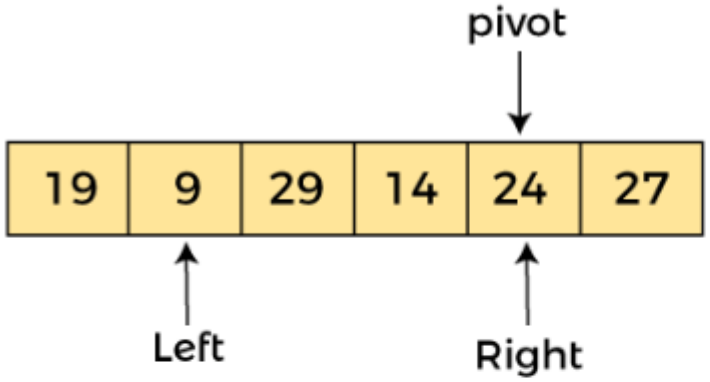
Because, a[pivot] > a[right], so, algorithm will swap a[pivot] with a[right], and pivot moves to right, as -



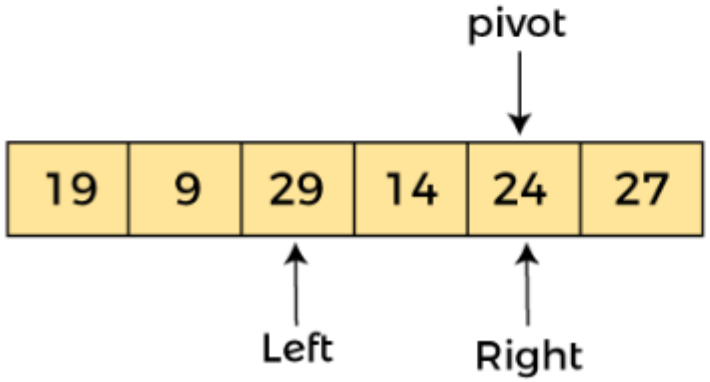
Now, a[left] = 19, a[right] = 24, and a[pivot] = 24. Since, pivot is at right, so algorithm starts from left and moves to right.

Quick Sort Algorithm

As a[pivot] > a[left], so algorithm moves one position to right as -

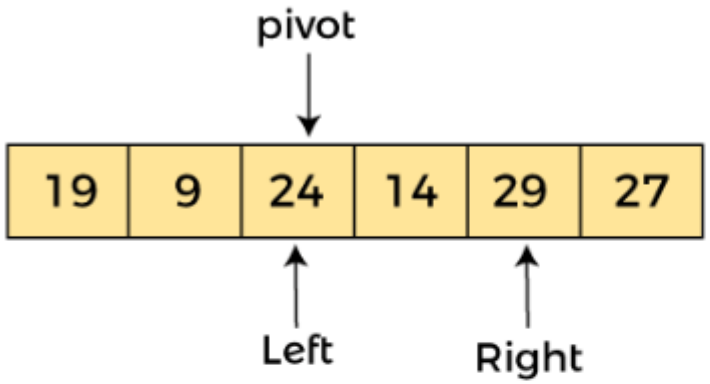


Now, a[left] = 9, a[right] = 24, and a[pivot] = 24. As a[pivot] > a[left], so algorithm moves one position to right as -

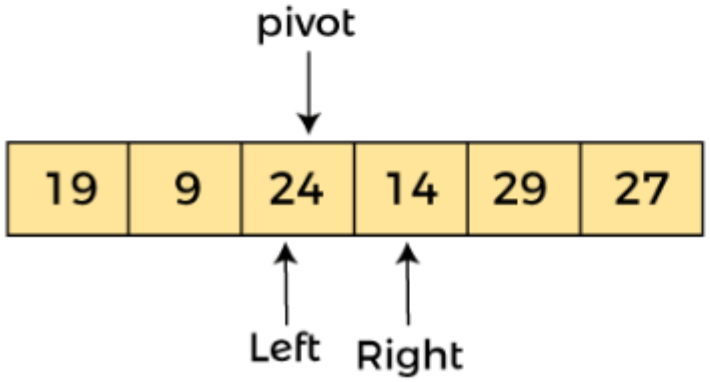


Quick Sort Algorithm

Now, a[left] = 29, a[right] = 24, and a[pivot] = 24. As a[pivot] < a[left], so, swap a[pivot] and a[left], now pivot is at left, i.e. -

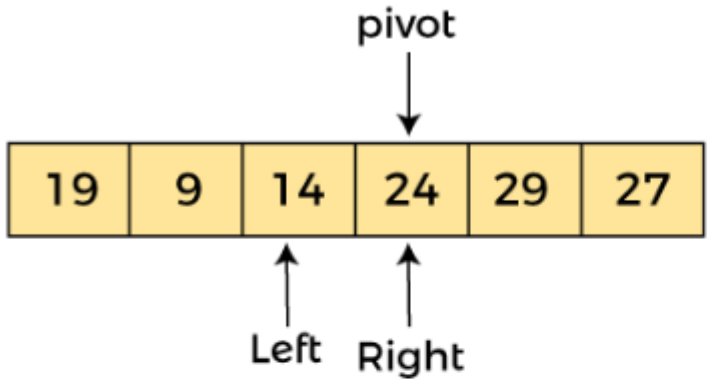


Since, pivot is at left, so algorithm starts from right, and move to left. Now, a[left] = 24, a[right] = 29, and a[pivot] = 24. As a[pivot] < a[right], so algorithm moves one position to left, as -

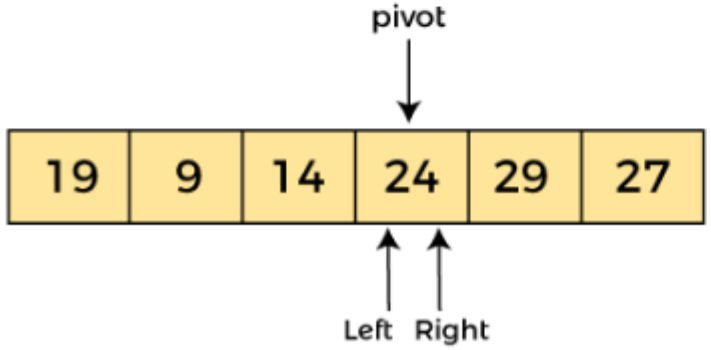


Quick Sort Algorithm

Now, a[pivot] = 24, a[left] = 24, and a[right] = 14. As a[pivot] > a[right], so, swap a[pivot] and a[right], now pivot is at right, i.e. -

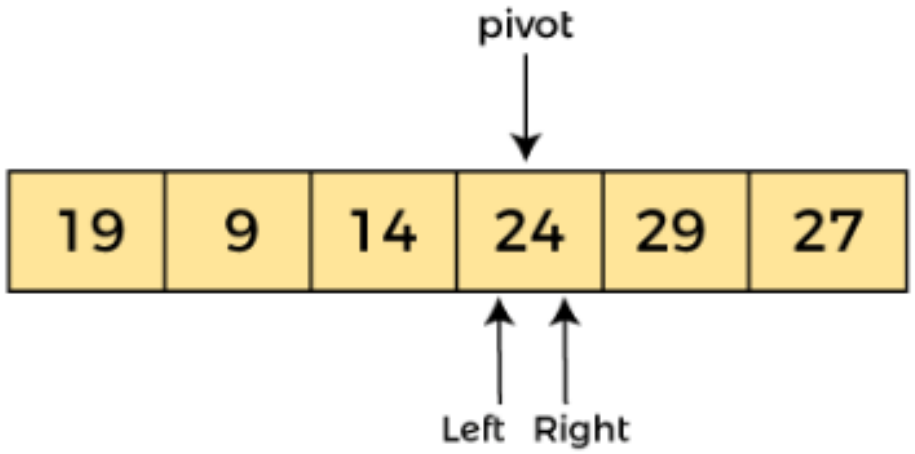


Now, a[pivot] = 24, a[left] = 14, and a[right] = 24. Pivot is at right, so the algorithm starts from left and move to right.



Quick Sort Algorithm

Now, a[pivot] = 24, a[left] = 14, and a[right] = 24. Pivot is at right, so the algorithm starts from left and move to right.

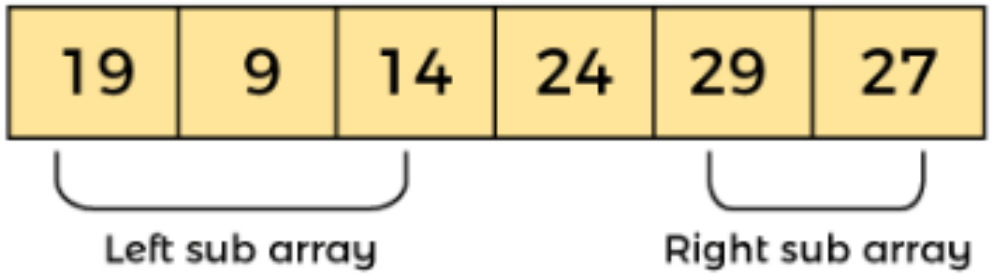


Now, a[pivot] = 24, a[left] = 24, and a[right] = 24. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

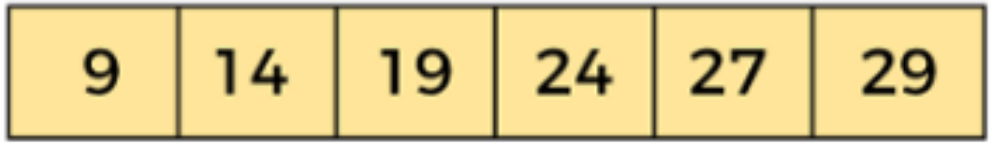
Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.

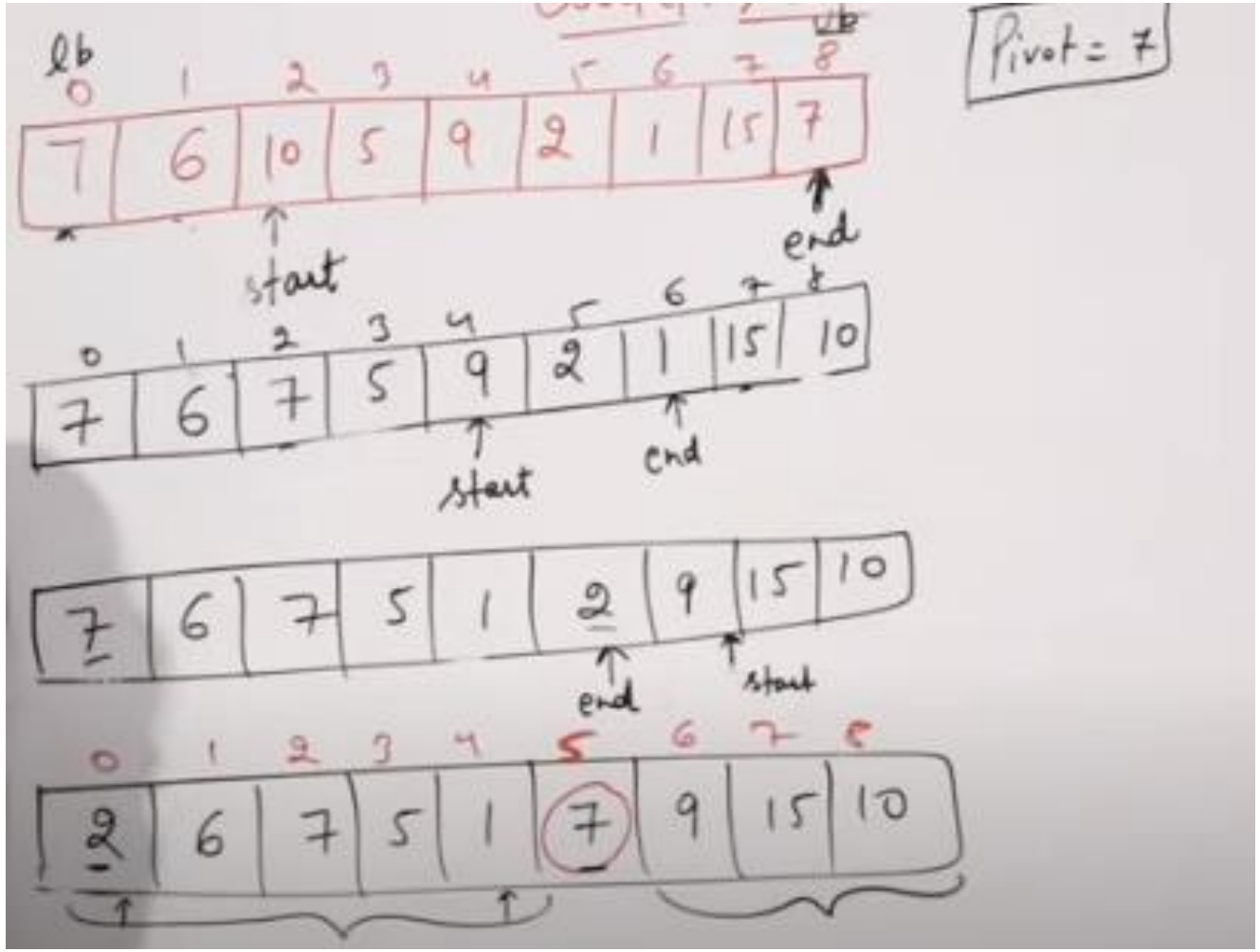
Quick Sort Algorithm



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -



Quick Sort Algorithm



Quicksort complexity

Now, let's see the time complexity of quicksort in best case, average case, and in worst case. We will also see the space complexity of quicksort.

1. Time Complexity

Case	Time Complexity
Best Case	O(n*logn)
Average Case	O(n*logn)
Worst Case	$O(n^2)$

- **Best Case Complexity -** In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is **O(n*logn)**.
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of quicksort is **O(n*logn)**.
- **Worst Case Complexity -** In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted already in ascending or descending order. The worst-case time complexity of quicksort is **$O(n^2)$**.

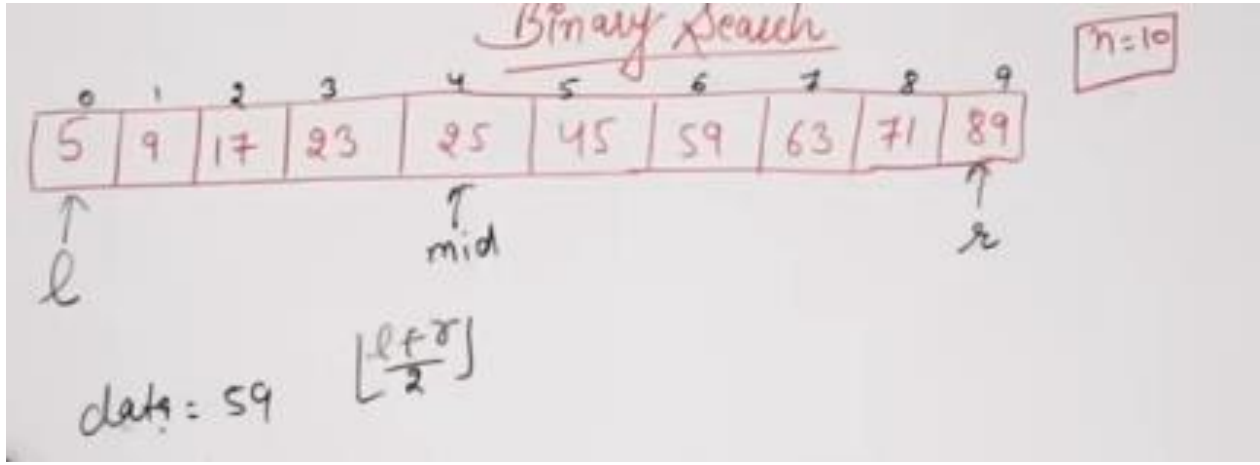
Binary Search Algorithm

- Binary search algorithm finds a given element in a list of elements with **O(log n)** time complexity where **n** is total number of elements in the list.
- The binary search algorithm can be used with only a sorted list of elements.
- That means the binary search is used only with a list of elements that are already arranged in an order.
- The binary search can not be used for a list of elements arranged in random order.
- This search process starts comparing the search element with the middle element in the list. If both are matched, then the result is "element found".
- Otherwise, we check whether the search element is smaller or larger than the middle element in the list.
- If the search element is smaller, then we repeat the same process for the left sublist of the middle element.
- If the search element is larger, then we repeat the same process for the right sublist of the middle element.
- We repeat this process until we find the search element in the list or until we left with a sublist of only one element.
- And if that element also doesn't match with the search element, then the result is "Element not found in the list".
- Time complexity in best case : search element is at mid, O(1)
- In worst case it is O(log n)

Binary Search Algorithm

- Binary search is implemented using following steps...
- **Step 1 -** Read the search element from the user.
- **Step 2 -** Find the middle element in the sorted list.
- **Step 3 -** Compare the search element with the middle element in the sorted list.
- **Step 4 -** If both are matched, then display "Given element is found!!!" and terminate the function.
- **Step 5 -** If both are not matched, then check whether the search element is smaller or larger than the middle element.
- **Step 6 -** If the search element is smaller than middle element, repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.
- **Step 7 -** If the search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.
- **Step 8 -** Repeat the same process until we find the search element in the list or until sublist contains only one element.
- **Step 9 - If that element also doesn't match with the search element, then display "Element is not found in the list!!!" and terminate the function.**

Binary Search Algorithm



Case I :- data == a[mid]

Case II :- data < a[mid]

Case III : data > a[mid]

Binary Search Algorithm

Example

Consider the following list of elements and the element to be searched...

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

search element **12**

Step 1:

search element (12) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
					12				

Both are not matching. And 12 is smaller than 50. So we search only in the left sublist (i.e. 10, 12, 20 & 32).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Binary Search Algorithm

Step 2:

search element (12) is compared with middle element (12)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
		12							

Both are matching. So the result is "Element found at index 1"

search element **80**

Step 1:

search element (80) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
					80				

Binary Search Algorithm

Both are not matching. And 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (80) is compared with middle element (65)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
							80		

Both are not matching. And 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Binary Search Algorithm

Step 3:

search element (80) is compared with middle element (80)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
								80	

Both are not matching. So the result is "Element found at index 7"

Hashing

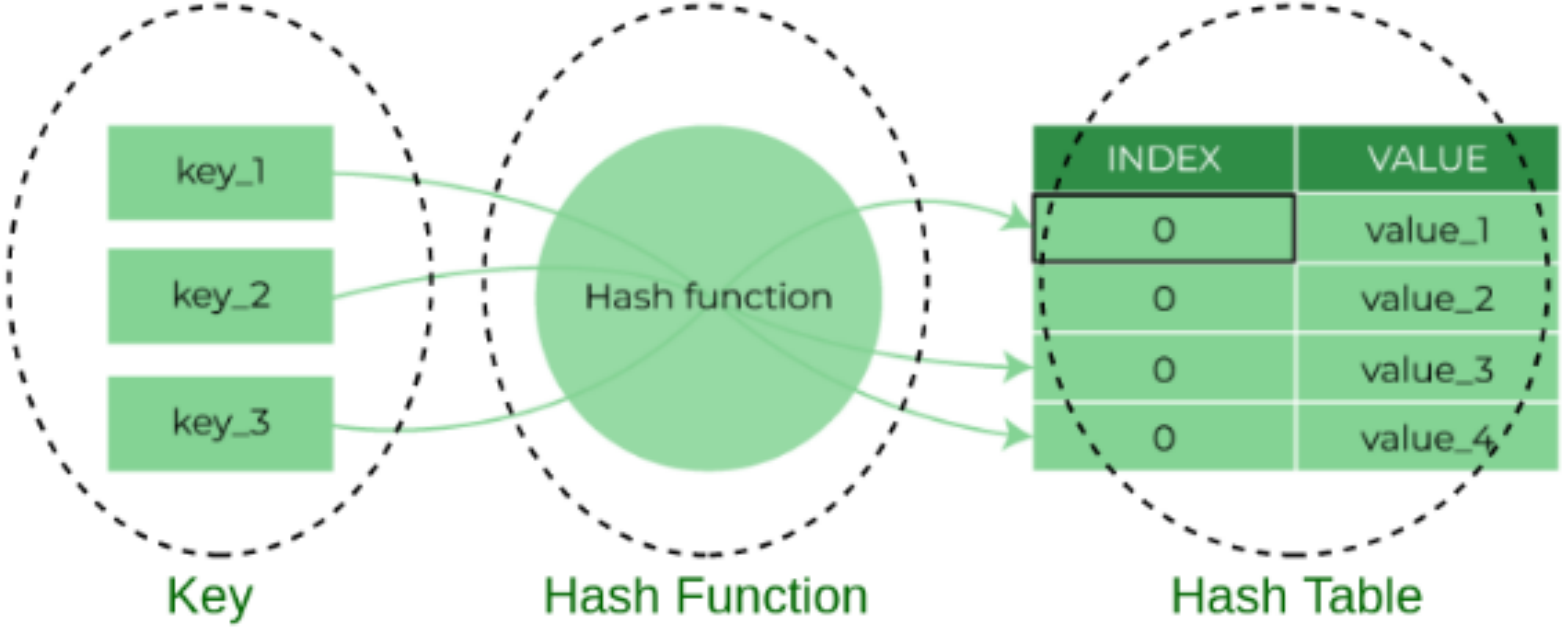
- In all search techniques like linear search, binary search and search trees, the time required to search an element depends on the total number of elements present in that data structure. In all these search techniques, as the number of elements increases the time required to search an element also increases linearly.

  Hashing is another approach in which time required to search an element doesn't depend on the total number of elements. Using hashing data structure, a given element is searched with **constant time complexity**. Hashing is an effective way to reduce the number of comparisons to search an element in a data structure.

Hashing

- Hashing is a technique or process of mapping keys, and values into the hash table by using a hash function.
- It is done for faster access to elements.
- The efficiency of mapping depends on the efficiency of the hash function used.
- Let a hash function H(x) maps the value **x** at the index **x%10** in an Array.
- For example if the list of values is [11,12,13,14,15] it will be stored at positions {1,2,3,4,5} in the array or Hash table respectively.

Hashing



Components of Hashing

For example key_1 is 52, hash function is k%10, 52%10 = 2, so 52 is stored at index 2 in hash table.
We can consider key as search key, which is going to be stored into hash table based on hash function used.
Then this hash table's value is linked with database where actual record is stored.

Hashing

Types of Hash Functions-

There are various types of hash functions available such as-
Mid Square Hash Function
Division Hash Function
Folding Hash Function etc

It depends on the user which hash function he wants to use.

Hash Table

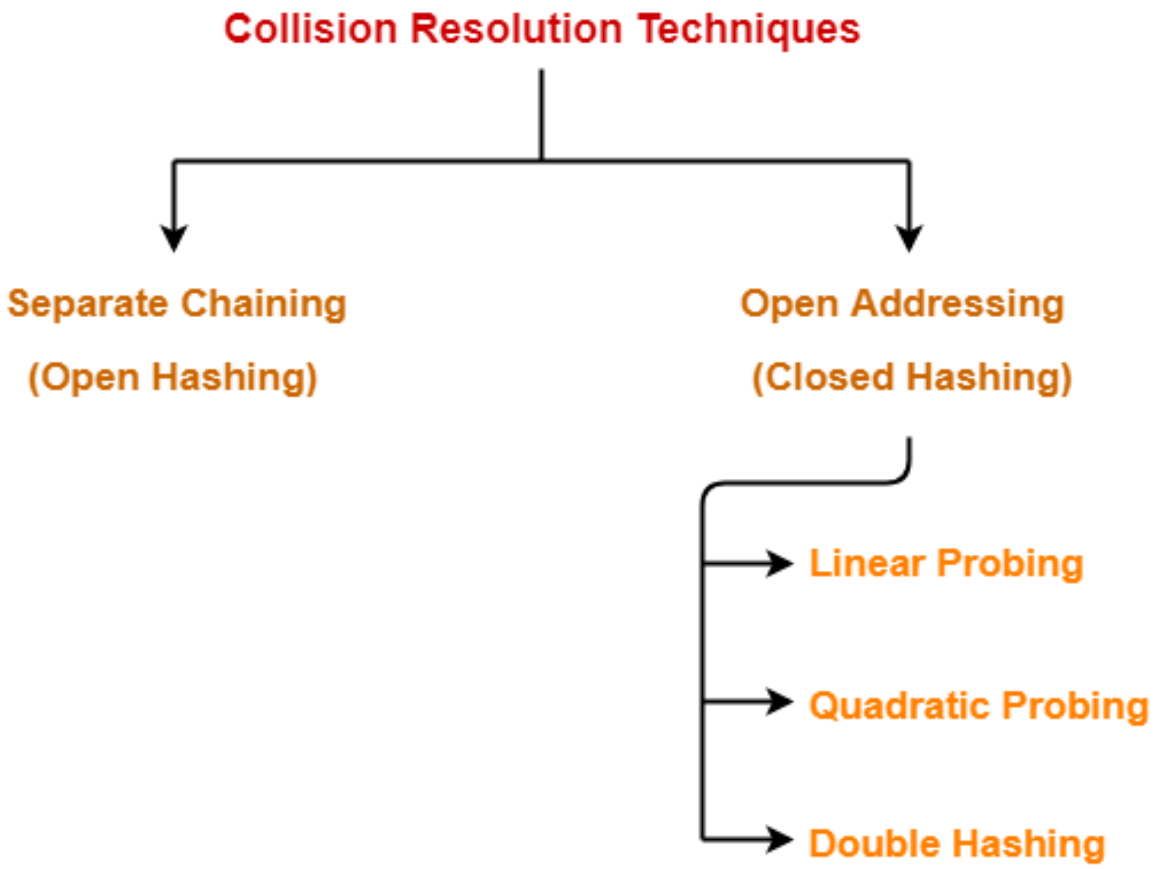
- Hash table is one of the most important data structures that uses a special function known as a hash function that maps a given value with a key to access the elements faster.
- A Hash table is a data structure that stores some information, and the information has basically two main components, i.e., key and value. The hash table can be implemented with the help of an associative array. The efficiency of mapping depends upon the efficiency of the hash function used for mapping.
- For example, suppose the key value is John and the value is the phone number, so when we pass the key value in the hash function shown as below:
- Hash(key)= index;

Collision

- When the hash value of a key maps to an already occupied bucket of the hash table, it is called as a **Collision**.
- When the two different values have the same value, then the problem occurs between the two values, known as a collision. In the above example, the value is stored at index 6. If the key value is 26, then the index would be:
- h(26) = 26%10 = 6
- Therefore, two values are stored at the same index, i.e., 6, and this leads to the collision problem. To resolve these collisions, we have some techniques known as collision techniques.
- The following are the collision techniques:
- Open Hashing: It is also known as closed addressing.
- Closed Hashing: It is also known as open addressing.

Collision resolution techniques

Collision Resolution Techniques are the techniques used for resolving or handling the collision.

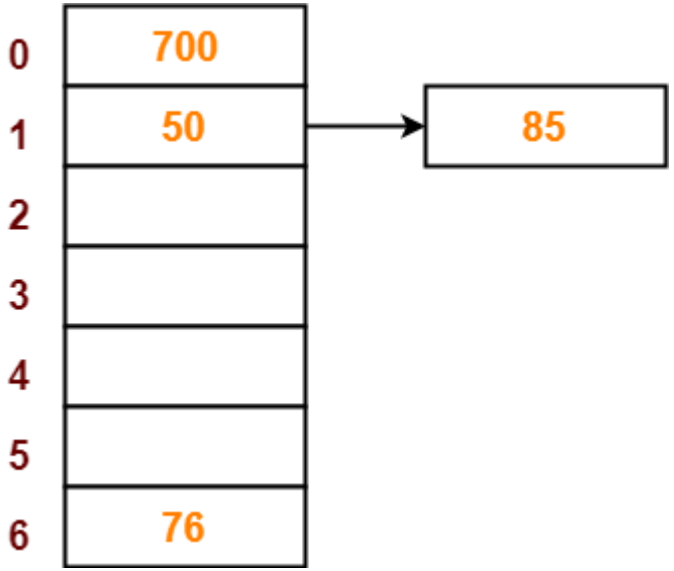


Collision resolution techniques

- **Separate chaining:** In this technique, a linked list is created from the slot in which collision has occurred, after which the new key is inserted into the linked list. This linked list of slots looks like a chain, so it is called **separate chaining**. It is used more when we do not know how many keys to insert or delete.

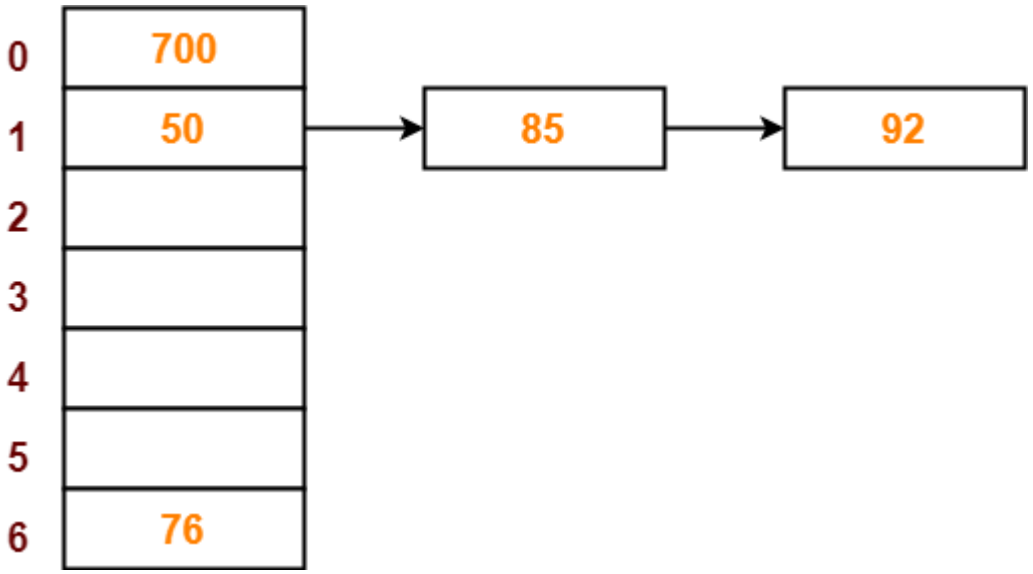
Collision resolution techniques

Example:
The next key to be inserted in the hash table = 85.
Bucket of the hash table to which key 85 maps = 85 mod 7 = 1.
Since bucket-1 is already occupied, so collision occurs.
Separate chaining handles the collision by creating a linked list to bucket-1.
So, key 85 will be inserted in bucket-1 of the hash table as-



Collision resolution techniques

- The next key to be inserted in the hash table = 92.
- Bucket of the hash table to which key 92 maps = 92 mod 7 = 1.
- Since bucket-1 is already occupied, so collision occurs.
- Separate chaining handles the collision by creating a linked list to bucket-1.
- So, key 92 will be inserted in bucket-1 of the hash table as-



Collision resolution techniques

- **<u>Open Addressing-</u>**

Open Addressing, which is also known as closed hashing is a technique of collision resolution in hash tables. The main idea of open addressing is to keep all the data in the same table to achieve it, we search for alternative slots in the hash table until it is found. Three well-known probe sequences are - Linear probing, quadratic probing, and double hashing.

- In open addressing,
- Unlike separate chaining, all the keys are stored inside the hash table.
- No key is stored outside the hash table.

Collision resolution techniques

- **Operations in Open Addressing-**
- Let us discuss how operations are performed in open addressing-

- **Insert Operation-**

- Hash function is used to compute the hash value for a key to be inserted.
- Hash value is then used as an index to store the key in the hash table.

- In case of collision,
- Probing is performed until an empty bucket is found.
- Once an empty bucket is found, the key is inserted.
- Probing is performed in accordance with the technique used for open addressing.

Collision resolution techniques

- **Operations in Open Addressing-**
- Let us discuss how operations are performed in open addressing-
- **Search Operation-**

- To search any particular key,
- Its hash value is obtained using the hash function used.
- Using the hash value, that bucket of the hash table is checked.
- If the required key is found, the key is searched.
- Otherwise, the subsequent buckets are checked until the required key or an empty bucket is found.
- The empty bucket indicates that the key is not present in the hash table.

Collision resolution techniques

- **<u>Operations in Open Addressing-</u>**
- Let us discuss how operations are performed in open addressing-
- **<u>Delete Operation-</u>**
-
- The key is first searched and then deleted.
- After deleting the key, that particular bucket is marked as “deleted”.
- During insertion, the buckets marked as “deleted” are treated like any other empty bucket.

Collision resolution techniques

- **<u>Open Addressing Techniques-</u>**
- Techniques used for open addressing are-
- **<u>1. Linear Probing-</u>**
-
- In linear probing,
- When collision occurs, we linearly probe for the next bucket.
- We keep probing until an empty bucket is found.

Collision resolution techniques

Example of linear probing -

Table Size = 7 Hash Function - $hash(key) = key\%7$ Collision Resoulution Strategy - $f(i) = i$

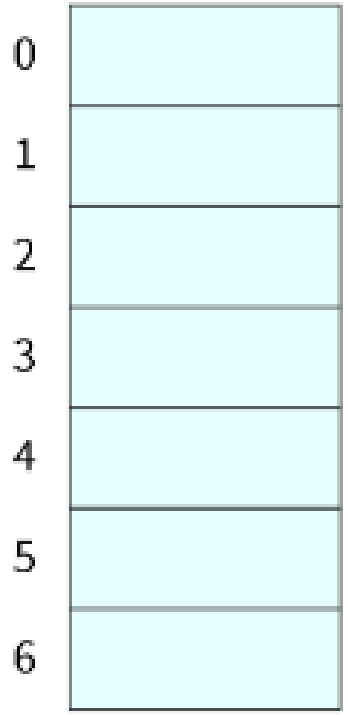
- Insert - $16, 40, 27, 9, 75$
- Search - $75, 21$
- Delete - 40

Steps involved are

Collision resolution techniques

- Step 1 - Make an empty hash table of size 7.

Step 1 -



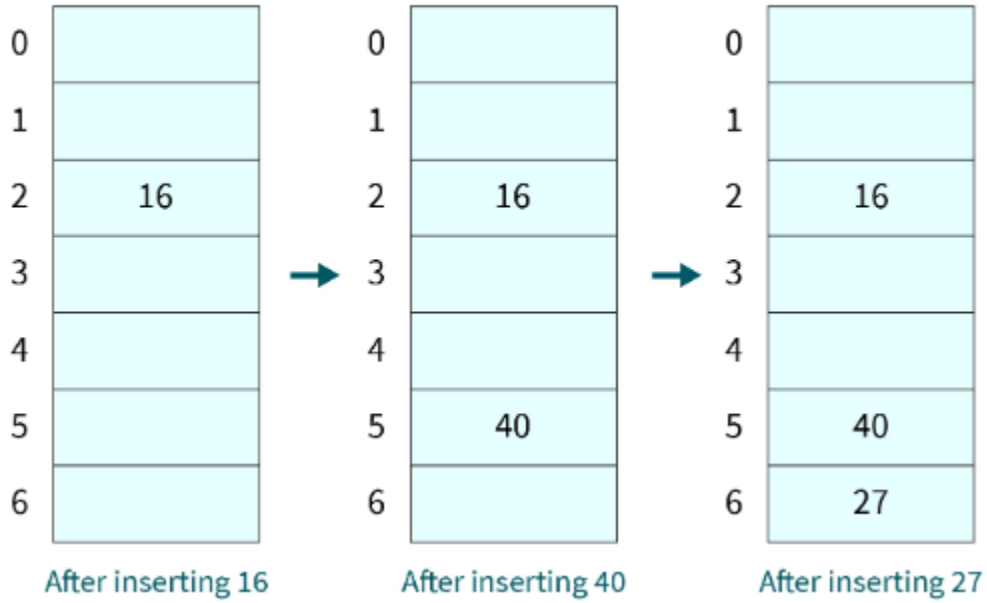
Collision resolution techniques

- Step 2 - Inserting 16, 40, and 27.
  - $hash(16) = 16\%7 = 2$
  - $hash(40) = 40\%7 = 5$
  - $hash(27) = 27\%7 = 6$

As we do not get any collision we can easily insert values at their respective indexes generated by the hash function.

After inserting, the hash table will look like -

Step 2 -



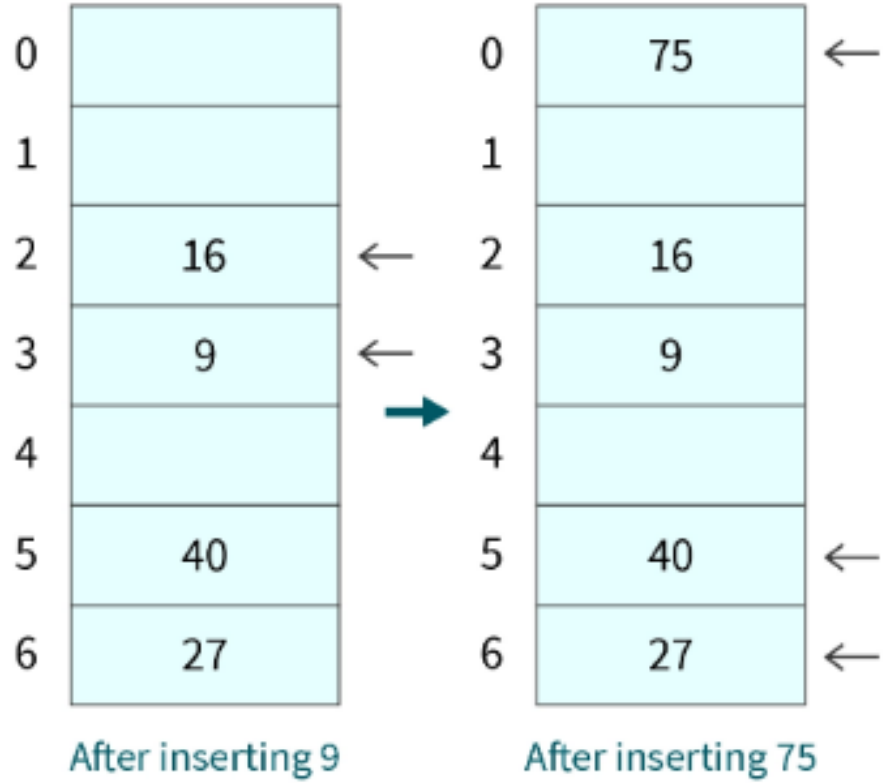
Collision resolution techniques

- Step 3 - Inserting 9 and 75.
  - $hash(9) = 9\%7 = 2$ But at index 2 already 16 is placed and hence collision occurs so as per linear probing we will search for consecutive cells till we find an empty cell.
    So we will probe for $hash(9) + 1$ $i.e.$ cell 3, since the next cell $i.e.$ 3 is not occupied we place 9 in cell 3.
  - $hash(75) = 75\%7 = 5$ Again collision happens because 40 is already placed in cell 5. So will search for the consecutive cells, so we search for cell 6 which is also occupied then we will search for cell $(hash(75) + 2)\%7$ $i.e.$ 0 which is empty so we will place 75 there.

After inserting 9 and 75 hash table will look like -

Collision resolution techniques

Step 3 -



Collision resolution techniques

Problem With Linear Probing

Even though linear probing is intuitive and easy to implement but it suffers from a problem known as *Primary Clustering*. It occurs because the table is large enough therefore time to get an empty cell or to search for a key k is quite large. This happens mainly because consecutive elements form a group and then it takes a lot of time to find an element or an empty cell which ultimately makes the worst case time complexity of *searching*, *insertion* and *deletion* operations to be $O(n)$, where n is the size of the table.

Collision resolution techniques

Quadratic Probing

Quadratic probing eliminates the problem of "Primary Clustering" that occurs in Linear probing techniques. The working of quadratic probing involves taking the initial hash value and probing in the hash table by adding successive values of an arbitrary quadratic polynomial. As suggested by its name, quadratic probing uses a quadratic collision function f. One of the most common and reasonable choices for f is -- $f(i) = i^2$ Operations in quadratic probing collision resolution strategy are -

- For inserting x we search for the cells $hash(x) + 0, hash(x) + 1^2, hash(x) + 2^2, \ldots$ until we find an empty cell to insert x.
- For searching x we again search for the cells $hash(x) + 0, hash(x) + 1^2, hash(x) + 2^2, \ldots$ until we find a cell with value x. If we find an empty cell that has never been occupied it means x is not present in the hash table.
- For deletion, we repeat the search process if a cell is found with value x we replace the value x with a predefined unique value to denote that this cell has contained some value in past.

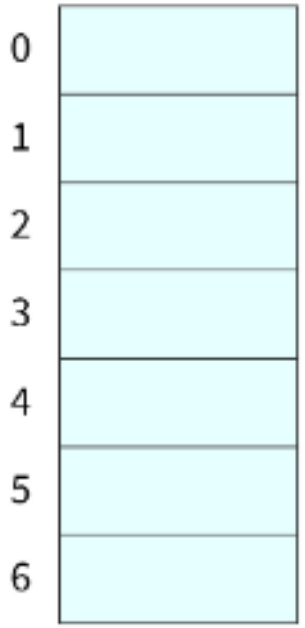
Collision resolution techniques

Example of Quadratic Probing

Table Size = 7 Insert = 15, 23, and 85. Search & Delete = 85 Hash Function - $Hash(x) = x\%7$ Collision Resolution Strategy - $f(i) = i^2$

- Step 1 - Create a table of size 7.

Step 1 -

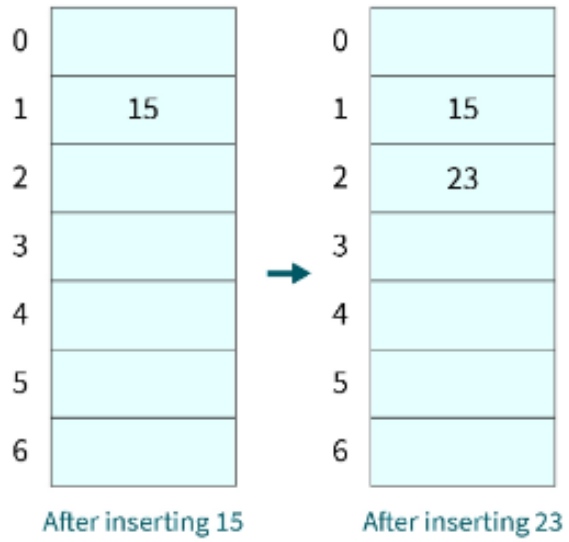


Collision resolution techniques

- Step 2 - Insert 15 and 23
  - $hash(15) = 15\%7 = 1$ Since the cell at index 1 is not occupied we can easily insert 15 at cell 1.
  - $hash(23) = 23\%7 = 2$ Again cell 2 is not occupied so place 23 in cell 2.

    After performing this step our hash table will look like -

Step 2 -



Collision resolution techniques

- Step 3 - Inserting 85
  - $hash(85) = 85\%7 = 1$ In our hash table cell 1 is already occupied so we will search for cell $1 + 1^2$, $i.e.$ cell 2. Again it is found occupied so we will search for cell $1 + 2^2$, $i.e.$ cell 5. It is not occupied so we will place 85 in cell 5. After performing all these 3 insertions in our hash table it will look like -

Step 3 -

0		
1	15	← 1 + 0
2	23	← 1 + 1²
3		
4		
5	85	← 1 + 2² = 5
6		

h(85) = 1

Collision resolution techniques

Problem with Quadratic Probing

It can be observed that when the hash value for two keys, say x_1 and x_2 is the same then they will follow the same probe sequence because $\forall k,\ h(x_1) + k^2 = h(x_2) + k^2$. Which leads to a milder form of clustering, called secondary clustering.

Collision resolution techniques

Double Hashing-

In double hashing,

•We use another hash function hash2(x) and look for i * hash2(x) bucket in i^{th} iteration.

•It requires more computation time as two hash functions need to be computed.

Collision resolution techniques

Example of double hashing

Table size = 7 $hash_1(k) = k\%7$ and $hash_2(k) = 1 + k\%6$ Insert = $37, 25, 12, 40,$ and 75 Search & Delete = 75

- Step 1 - Create a table of size 7.

Step 1 -

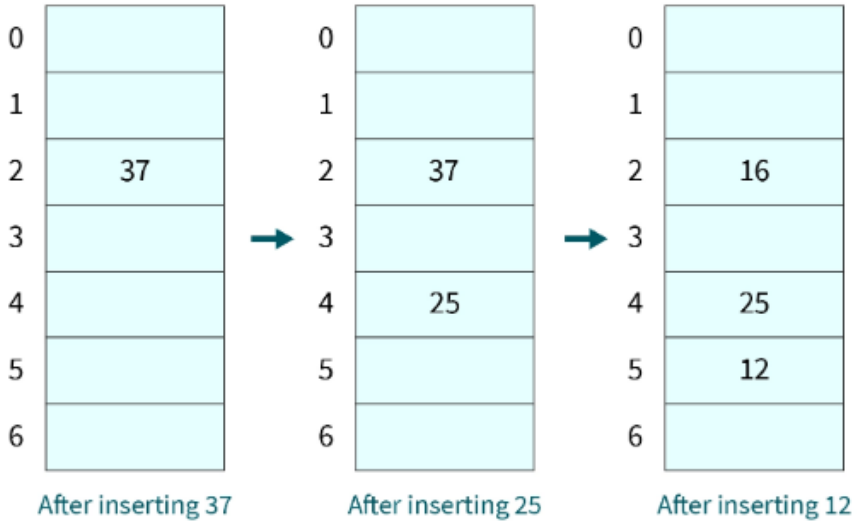
0	
1	
2	
3	
4	
5	
6	

Collision resolution techniques

- Step 2 - Insert 37, 25, and 12.
  - $hash_1(37) = 37\%7 = 2$
  - $hash_1(25) = 25\%7 = 4$
  - $hash_1(12) = 12\%7 = 5$

  There is no collision at any point during inserting these three values so we will simply place these elements at their respective positions. After which, our hash table will look like -

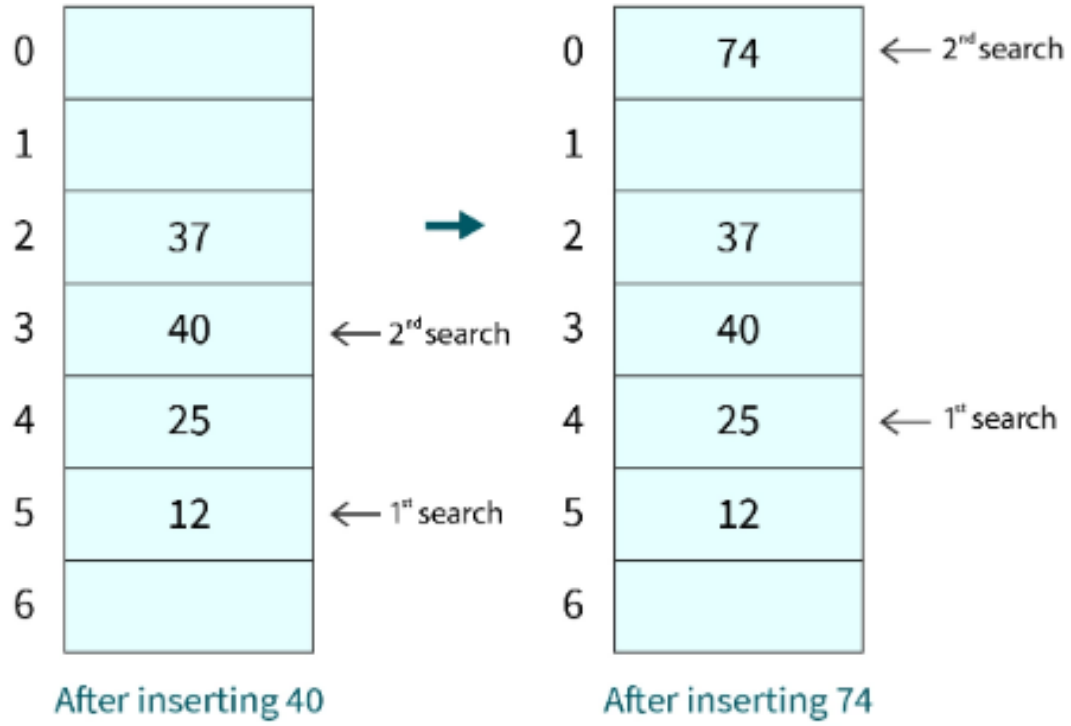
Step 2 -



Collision resolution techniques

- Step 3 - Insert 40 and 74
  - $hash_1(40) = 40\%7 = 5, \quad hash_2(40) = 1 + 40\%6 = 5$ But at the cell at index 5 is already occupied, so we will check for next cell $i.e.$ $hash_1(40) + 1 \times hash_2(40)$ which will evaluate to $(5 + 5)\%7 = 3$. Cell at index 3 is not occupied so we will place 40 in cell 3.
  - $hash_1(74) = 74\%7 = 4, \quad hash_2(74) = 1 + 74\%6 = 3$ But at the cell at index 4 is already occupied, so we will check for next cell $i.e.$ $hash_1(74) + 1 \times hash_2(74)$ which will evaluate to $(4 + 3)\%7 = 0$. Cell at index 0 is not occupied so we will place 74 in cell 0.

After which our hash table will look like -

Collision resolution techniques

Step 3 -